

ENTRADE.MA

Rapport de Soutenance Chatbot GraphRAG

*Conception, mise en production et amélioration continue
d'un assistant conversationnel bilingue (AR/FR)*

Pipeline, architecture, incidents réels et solutions – de la première ingestion de données à la mémoire conversationnelle

Périmètre du document

Architecture système à deux VMs (inférence CPU + stockage/orchestration) • Pipeline d'ingestion et de graphe de connaissances (Neo4j/Qdrant) • Pipeline RAG applicatif complet (Router, Planner, Fusion, Reranking, Guardrail) • Historique détaillé des incidents rencontrés et de leurs corrections • Mémoire conversationnelle • Méthodologie de test et de validation • Limitations connues et recommandations

Projet	ENTRADE.MA – Assistant GraphRAG pour l'Entraide Nationale
Modèle LLM	Qwen2.5-7B-Instruct Q4_K_M (llama.cpp, inférence CPU)
Embeddings	BGE-M3 (dense 1024D) + BGE-Reranker-v2-m3 (cross-encoder)
Bases	Neo4j (graphe métier) • Qdrant (vectoriel hybride) • Redis (cache & mémoire de session)
API	FastAPI – pipeline Router / Planner / Executor / Fusion / Reranker / Guardrail

Table des matières

1	Résumé exécutif	2
2	Contexte et objectifs	2
3	Architecture générale	3
3.1	Infrastructure à deux machines	3
3.2	Le pipeline RAG applicatif	3
4	Pipeline de données (ETL et graphe de connaissances)	6
4.1	Schéma du graphe Neo4j	7
4.2	Schéma vectoriel Qdrant	7
5	Historique détaillé des incidents et de leurs résolutions	7
5.1	Mémoire conversationnelle	12
5.2	Comparaison de modèles (Qwen2.5-7B contre Gemma-3-4B)	13
5.3	Réponse de repli inutilisable pour les foires aux questions et les programmes	13
6	Méthodologie de test et de validation	14
7	Limitations connues et recommandations	14
8	Conclusion	15

1 Résumé exécutif

ENTRADE.MA est un assistant conversationnel bilingue (arabe et français) destiné à orienter les citoyens vers les centres, services et programmes de l'Entraide Nationale du Maroc. Le système repose sur une architecture GraphRAG (Retrieval-Augmented Generation appuyée sur un graphe de connaissances) : les réponses ne sont pas produites librement par un grand modèle de langage, mais construites à partir de données réellement présentes dans deux bases – un graphe Neo4j qui modélise les relations métier (régions, délégations, communes, centres, services, populations cibles, programmes, institutions, foire aux questions) et une base vectorielle Qdrant qui permet une recherche sémantique hybride sur le même corpus. Le modèle de langage, Qwen2.5-7B-Instruct, est exécuté localement via llama.cpp sur un serveur CPU dédié, sans dépendance à une API externe – un choix structurant pour la souveraineté des données et le coût d'exploitation, mais qui impose en retour des contraintes de latence fortes qui ont guidé une bonne partie des décisions d'ingénierie décrites dans ce rapport.

Le projet s'est déroulé en deux temps. Une première phase, documentée dans le *Guide de Déploiement* de référence, a posé l'infrastructure : provisionnement des deux machines virtuelles, installation de la couche d'inférence, installation de la couche de stockage, mise en place de l'API d'orchestration, sécurisation réseau, frontend et observabilité. La seconde phase, objet principal du présent rapport, a consisté à faire passer le système d'un prototype fonctionnel à un assistant réellement fiable : ingestion complète et vérifiée des données métier, audit exhaustif de la couverture des données, refonte du pipeline de récupération et de génération pour qu'il réponde *prioritairement* à partir de la base de connaissances tout en sachant reconnaître honnêtement les questions hors domaine, correction méthodique d'une série de défauts réels découverts par des tests de bout en bout (mise en cache inopérante, filtre de pertinence structurellement défaillant, couverture Centre-Service incomplète pour deux catégories de population, ancrage géographique insuffisant, fuite de caractères parasites du modèle, contrôle de langue trop permissif), puis ajout d'une mémoire conversationnelle pour que les questions de suivi elliptiques restent comprises dans leur contexte réel.

Chaque intervention décrite dans ce document a suivi la même discipline : reproduire le défaut sur une vraie question, en diagnostiquer la cause réelle par inspection directe des données (jamais par supposition), appliquer un correctif ciblé, puis vérifier le résultat par un nouveau test réel avant de considérer le problème clos. Cette discipline, et son historique détaillé incident par incident, constitue le coeur de ce rapport.

2 Contexte et objectifs

L'Entraide Nationale gère, sur l'ensemble du territoire marocain, un réseau dense de centres (crèches, centres d'éducation et de formation, centres d'accompagnement pour la protection de l'enfance, établissements de protection sociale, centres pour personnes âgées ou en situation de handicap, etc.), chacun proposant un sous-ensemble de services soumis à des conditions et des documents précis, et rattaché à des programmes institutionnels plus larges. Cette information est par nature fragmentée : elle vit dans des fiches, des foires aux questions et des annuaires hétérogènes, difficiles à interroger rapidement pour un citoyen qui cherche une réponse concrète ("où", "sous quelles conditions", "avec quels documents").

L'objectif du projet est de mettre cette information à disposition à travers une interface conversationnelle simple – un widget de chat intégrable sur un site – capable de répondre en arabe ou en français, en citant systématiquement une source vérifiable, plutôt que de produire une réponse plausible mais non garantie. Trois exigences ont structuré l'ensemble des choix techniques :

- **Exactitude avant tout.** Le système ne doit jamais inventer une condition, un document, un centre ou un montant : chaque affirmation dans une réponse doit être traçable jusqu'à une donnée réellement présente dans Neo4j ou Qdrant. Quand l'information n'existe pas, le système doit le dire, plutôt que de improviser.
- **Bilinguisme réel, pas une traduction de façade.** Les citoyens posent leurs questions indifféremment en arabe standard, en darija transcrite en caractères arabes, ou en français – parfois de façon très elliptique. Le système doit reconnaître l'intention et répondre dans la langue de la question, sans mélange.
- **Fonctionnement 100% local, sur CPU.** Pour des raisons de souveraineté des données (informations sur des populations vulnérables) et de maîtrise des coûts, aucun appel à une API de modèle de langage externe n'est autorisé : l'inférence tourne sur un serveur CPU dédié via llama.cpp. Cette contrainte, plus que toute autre, a orienté les arbitrages de performance décrits dans ce rapport – chaque génération ancrée sur le contexte prend, selon la charge, de 60 à plus de 300 secondes, ce qui interdit les architectures qui multiplient les appels au modèle sans discernement.

Le présent rapport documente la manière dont ces trois exigences ont été traduites en architecture, puis en code, puis vérifiées – non pas comme un exercice théorique, mais à travers une série de tests réels menés tout au long du projet, chacun ayant révélé un défaut concret et donné lieu à un correctif vérifié.

3 Architecture générale

3.1 Infrastructure à deux machines

Le système est déployé sur deux machines virtuelles aux rôles strictement séparés, conformément au *Guide de Déploiement* de référence :

- **VM1 – couche d'inférence.** Héberge exclusivement le serveur llama.cpp et le modèle Qwen2.5-7B-Instruct quantifié (Q4_K_M), exposé en HTTP compatible OpenAI. Paramètres de lancement retenus : `-t 10 -c 8192 -ngl 0 -temp 0.3 -top-p 0.9 -repeat-penalty 1.1 -n 512 -parallel 2 --ngl 0` signifiant une inférence 100% CPU, `-parallel 2` autorisant deux générations concurrentes sur le même processus.
- **VM2 – couche de stockage et d'orchestration.** Héberge Neo4j (graphe de connaissances), Qdrant (recherche vectorielle hybride), Redis (cache et, depuis la dernière itération, mémoire de session) ainsi que l'API FastAPI qui orchestre l'ensemble du pipeline applicatif décrit ci-dessous, et le widget frontend.

Deux scripts d'orchestration (`scripts/start_all.sh` et `scripts/stop_all.sh`) pilotent le cycle de vie de la couche applicative (llama.cpp, API, serveur du widget) indépendamment des conteneurs de stockage (Neo4j/Qdrant/Redis, gérés séparément par docker compose avec redémarrage automatique). Ce découplage s'est révélé important en pratique : à plusieurs reprises pendant le développement, la couche applicative s'est arrêtée entre deux sessions de travail sans que les conteneurs de stockage soient affectés, ce qui a permis un redémarrage rapide sans perte de données – ce point est repris en section 7 comme axe d'amélioration (absence de superviseur de processus).

3.2 Le pipeline RAG applicatif

La figure 1 présente le cheminement complet d'une requête utilisateur, de sa réception par l'API jusqu'à la réponse renvoyée. Cette architecture a été explicitement alignée, en cours de

projet, sur un schéma cible fourni par le porteur du projet, puis affinée à chaque défaut découvert en test réel.

Chaque étage a un rôle et des entrées/sorties précis, détaillés ci-dessous.

Query Preprocessor. Normalise la requête (une seule forme canonique pour deux lettres arabes visuellement proches mais distinctes – le tâ marbûta et le hâ) pour la détection du calculeur et la clé de cache – **mais pas** pour le routage sémantique lui-même, qui reste appliqué au texte original (voir section 5, l’un des défauts corrigés vient précisément d’une normalisation trop agressive appliquée au mauvais endroit).

Calculator. Détecte une opération arithmétique simple (“combien font 145 plus 320”) et répond par logique métier directe, sans passer ni par la récupération ni par le modèle de langage – un choix économique : sur un modèle exécuté en CPU à quelques tokens par seconde, il est absurde de payer 60 à 100 secondes de génération pour une addition.

Router. Classifie l’intention (recherche de centre, de service, de FAQ, de programme 2027, question sur l’institution, question ouverte, ou simple salutation), détecte la langue et extrait les entités métier reconnues dans le texte (types de population, institutions, régions) par un ensemble de motifs lexicaux bilingues.

Planner / Executor. Le Planner transforme l’intention et les entités en un plan d’appel d’outils (recherche vectorielle Qdrant, expansion de graphe Neo4j, ou les deux en vagues successives) ; l’Executor l’exécute réellement. La recherche Qdrant est *hybride* – un vecteur dense (BGE-M3, 1024 dimensions, similarité cosinus) combiné à un vecteur creux (TF-IDF haché) sur la même collection nommée. L’expansion de graphe part des identifiants métier retournés par la recherche vectorielle (jamais des identifiants internes du point Qdrant) pour parcourir les relations pertinentes dans Neo4j.

Fusion & Ranking. Combine les deux sources de résultats par une formule métier explicite : $0,35 \times \text{RRF} + 0,25 \times \text{OFFRE} + 0,25 \times \text{population} + 0,15 \times \text{géographie}$, assortie de pénalités multiplicatives (0,70 pour un établissement de protection sociale proposé comme dernier recours, 0,85 quand la population cible n’est pas renseignée, 0,55 – pénalité plus sévère – quand la population renseignée contredit explicitement celle demandée). Ces poids ont été recalibrés en cours de projet après qu’un diagnostic sur une requête réelle a montré qu’un rang vectoriel brut trop bruité sur des requêtes arabes courtes pouvait faire dominer un résultat hors-sujet : le signal population, déterministe et tiré directement des données, s’est révélé plus fiable que le rang vectoriel seul.

Reranker. Un cross-encoder (BGE-Reranker-v2-m3) réévalue sémantiquement les meilleurs candidats issus de la fusion – non pas l’ensemble des résultats fusionnés (mesuré à environ 0,5 seconde par candidat sur ce CPU), mais un sous-ensemble déjà filtré par le score métier, ce qui réduit le temps de reranking d’un facteur proche de 3 sans perte de qualité mesurée.

Relevance Gate. Décide si les résultats récupérés sont réellement pertinents avant de les transmettre au modèle de langage. Ce filtre applique deux régimes distincts, affinés après deux faux-négatifs observés en test réel (détaillés en section 5) : pour les quatre intentions étroitement reconnues par un motif métier explicite (recherche de centre, de service, de FAQ, de programme), le motif d’activation lui-même fait foi de pertinence ; pour les deux intentions

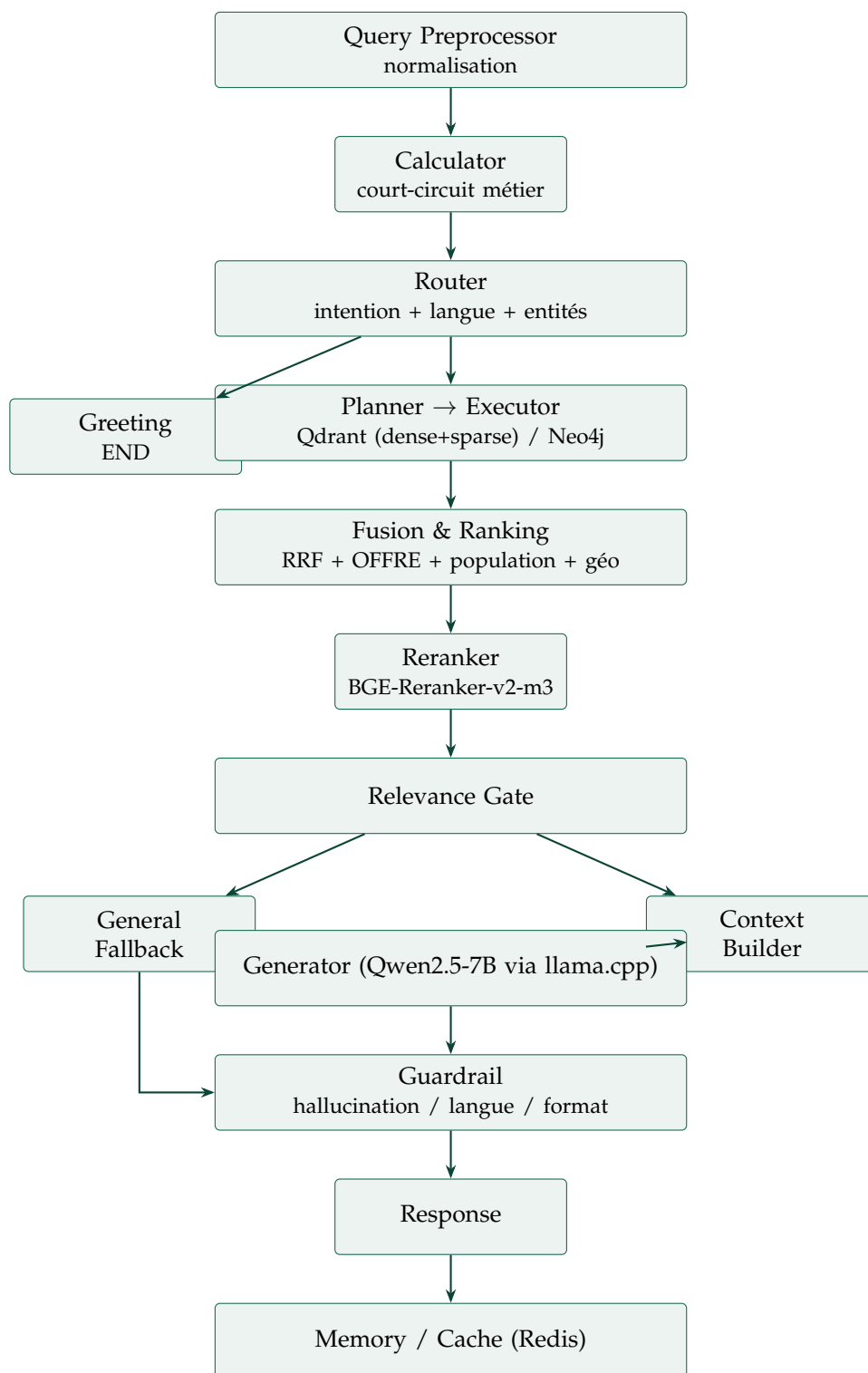


FIGURE 1 – Pipeline RAG applicatif – de la requête à la réponse mise en cache. La branche *Greeting* (salutation détectée par le Router) court-circuite l’ensemble du pipeline de récupération et de génération pour aboutir directement à *Response*, sans passer par le modèle de langage.

sans preuve préalable (question ouverte, question sur l'institution), un seuil strict s'applique, combinant le score du reranker et la présence d'entités métier extraites.

Context Builder / General Fallback. Si les résultats sont jugés pertinents, le Context Builder structure un contexte riche (jusqu'à six résultats, avec conditions, documents, budget, institutions) destiné au modèle de langage. Sinon, la branche *General Fallback* autorise le modèle à répondre à partir de ses connaissances générales, mais avec un préfixe de transparence fixe (non laissé à la discrétion du modèle) indiquant explicitement que la réponse ne provient pas de la base de données de l'Entraide Nationale.

Generator. Qwen2.5-7B-Instruct, appelé via llama.cpp, produit la réponse à partir du contexte structuré et d'un prompt système qui interdit explicitement toute invention de condition, de document, de centre ou de programme.

Guardrail. Contrôle la réponse générée avant qu'elle ne parte au client : détection d'affirmations numériques ou de codes d'institution non cités dans le contexte, ancrage thématique (recouvrement lexical avec le contexte), cohérence de langue (y compris une détection spécifique de caractères chinois parasites, voir section 5), et format (absence de balisage *markdown*, la réponse devant rester du texte brut lisible dans le widget). En cas d'invalidation, une réponse de repli structurée est renvoyée plutôt qu'une erreur.

Memory / Cache. Deux niveaux de cache Redis coexistent : un cache de contexte de récupération, et un cache de réponse finale déjà générée. Depuis la dernière évolution du projet, cette étape intègre également la mémoire conversationnelle par session, détaillée en section 5.1.

4 Pipeline de données (ETL et graphe de connaissances)

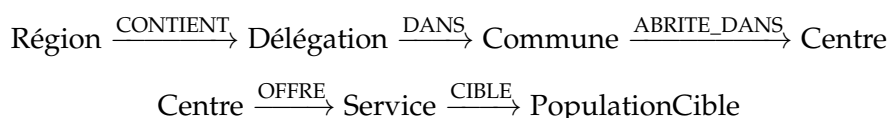
En amont du pipeline applicatif décrit ci-dessus, un pipeline de préparation des données transforme les sources brutes (fiches de centres, tableaux de services, foire aux questions, fiches de programmes) en deux bases interrogeables. Ce pipeline est organisé en sept carnets Jupyter exécutés dans un ordre strict :

Étape	Rôle
00_Utils	Fonctions communes (nettoyage de texte, journalisation, accès aux chemins de données).
01_Normalize	Nettoyage et normalisation des champs bruts (dates, codes d'institution, libellés de population).
02_Chunk	Découpage du contenu en fragments indexables, avec un texte concaténé propre à chaque type d'entité (par exemple, pour une FAQ, la question et la réponse concaténées).
03_Match	Résolution des relations métier implicites, en particulier l'appariement Centre ↔ Service par population cible – étape à l'origine de l'un des incidents les plus significatifs du projet (section 5).
04_Embed	Calcul des vecteurs denses (BGE-M3) pour chaque fragment.
05_Neo4j_Load	Chargement du graphe métier dans Neo4j.
06_Qdrant_Index	Indexation vectorielle hybride dans Qdrant.

Un point de conception important, qui a conditionné la méthode de correction de chaque défaut de données rencontré : les carnets 05 et 06 sont *destructifs et idempotents*. Le carnet 05 exécute un `MATCH (n) DETACH DELETE n` avant de recharger l’intégralité du graphe ; le carnet 06 supprime puis recrée la collection Qdrant avant réindexation. Cette conception garantit qu’un rechargement complet reproduit toujours le même état, mais implique en contrepartie qu’*aucune correction de données ne doit être appliquée directement en base* – toute correction doit remonter jusqu’aux fichiers intermédiaires (`data/processed/*.jsonl`) puis être rejouée à travers le pipeline complet, sous peine d’être silencieusement effacée au prochain rechargement. Cette règle a été respectée pour l’ensemble des corrections de données décrites dans ce rapport.

4.1 Schéma du graphe Neo4j

Le graphe métier modélise la hiérarchie administrative et l’offre de services :



auquel s’ajoutent des noeuds Institution (types d’établissement, par exemple CEF, CPSH, EPS), Programme (les grands programmes institutionnels, avec leur budget) et FAQ (questions-réponses génériques sur la mission de l’Entraide Nationale, avec des relations CIBLE/CONCERNE vers les populations et institutions concernées).

4.2 Schéma vectoriel Qdrant

Une collection unique (`entraide_mvp`) porte deux vecteurs nommés par point : un vecteur dense (`BGE-M3`, 1024 dimensions, distance cosinus) et un vecteur creux sparse (TF-IDF haché), ce qui permet une recherche hybride combinant proximité sémantique et correspondance lexicale exacte – utile notamment pour les identifiants d’institution (CEF, CPSH...) que la seule similarité sémantique retrouve mal. Chaque point porte, dans son *payload*, le type et l’identifiant métier de l’entité (partagé avec Neo4j, ce qui permet de faire le pont entre recherche vectorielle et expansion de graphe), le type de fragment, la population ciblée, les institutions concernées, la région, la langue, et un indicateur de dernier recours pour les établissements de protection sociale.

5 Historique détaillé des incidents et de leurs résolutions

Cette section constitue le cœur technique du présent rapport. Chaque incident décrit ci-dessous a été découvert par un test réel (jamais anticipé sur le papier), diagnostiqué par inspection directe des données ou du comportement du système, corrigé, puis re-vérifié par un nouveau test réel avant d’être considéré résolu. Cette discipline – ne jamais corriger sur la base d’une supposition, toujours vérifier sur des données réelles avant et après – a été appliquée de façon constante tout au long du projet.

Incident – Cache de réponse Redis écrit mais jamais relu

Symptôme observé. Une question déjà posée à l’identique redéclenchait systématiquement une génération complète du modèle de langage (60 à 160 secondes), alors qu’un mécanisme de cache existait déjà dans le code.

Diagnostic. Inspection du code de l’API : la fonction `cache.set(...)` était bien appelée à quatre endroits différents après chaque génération de réponse, mais aucune lecture correspondante (`cache.get(...)`) n’existait avant de lancer une nouvelle génération. Le cache

accumulait des entrées sans jamais être consulté – un cache en écriture seule.

Solution appliquée. Ajout d’une vérification `cache.get(normalized_query, query_type="answer")` au tout début des points d’entrée `/chat` et `/chat/stream`, avant même le calculateur métier. Toutes les écritures de cache ont été unifiées pour stocker un objet structuré cohérent (réponse, langue, intention, sources) sous une clé simplifiée, identique entre écriture et lecture.

Justification. Le coût de cette vérification est négligeable (une lecture Redis) comparé au coût d’une génération complète sur ce CPU ; il n’y avait aucune raison de ne pas la faire systématiquement, y compris avant le calculateur.

Vérification. Rejouer la même question deux fois de suite : la seconde réponse est désormais immédiate (temps de récupération et de génération à zéro), avec un contenu identique à la première.

Incident – Filtre de pertinence structurellement défaillant pour les questions ouvertes

Symptôme observé. Une question clairement hors du domaine de l’Entraide Nationale (“quelle est la capitale du Japon ?”) obtenait malgré tout dix résultats de recherche vectorielle, avec un score final atteignant 0,54 – suffisant pour être présenté comme une donnée pertinente de la base.

Diagnostic. Le filtre de pertinence pour l’intention *question ouverte* s’appuyait sur la condition `result_count == 0`. Or une recherche vectorielle non filtrée par type d’entité renvoie par construction toujours les *plus proches voisins*, même sans rapport réel avec la question : cette condition ne pouvait structurellement jamais être vraie.

Solution appliquée. Remplacement du critère par un seuil sur le score du reranker (cross-encoder), mesuré à 0,94–0,99 sur de vraies correspondances contre 0,0 sur le cas “capitale du Japon” – une marge large et fiable. Ce seuil a ensuite été affiné (voir l’incident suivant) après deux faux-négatifs découverts en généralisant ce filtre à d’autres intentions.

Justification. Contrairement au nombre de résultats, le score du reranker mesure une correspondance sémantique réelle entre la question et chaque passage candidat : c’est le seul signal disponible qui discrimine correctement une question hors-domaine d’une question pertinente sans résultat vectoriel filtré.

Vérification. La question sur la capitale du Japon obtient désormais une réponse de connaissance générale avec mention explicite qu’elle ne provient pas de la base de données, tandis que des questions pertinentes réelles continuent d’obtenir une réponse ancrée sur le contexte.

Incident – Généralisation du filtre de pertinence trop agressive

Symptôme observé. Après avoir étendu le filtre de pertinence strict (score du reranker) à toutes les intentions de domaine, deux questions pourtant légitimes ont été rejetées à tort : une recherche de crèche à Salé et une question FAQ sur l’aide aux oeuvres sociales obtenaient toutes deux un score de reranker de 0,0, identique au cas hors-domaine.

Diagnostic. Le texte indexé pour les centres est en grande partie une adresse administrative en français/latin ; face à une question posée en arabe, la correspondance cross-linguale du reranker s’effondre même sur un résultat pertinent. Le score du reranker seul n’est donc pas un signal fiable pour ce type de contenu.

Solution appliquée. Le filtre de pertinence strict n’est finalement conservé que pour les deux intentions qui ne bénéficient d’aucune preuve de pertinence préalable (*question ouverte*, *question sur l’institution*). Pour les quatre intentions étroitement reconnues par un

motif métier explicite (recherche de centre, de service, de FAQ, de programme), le motif d’activation lui-même fait foi – le garde-fou de sécurité restant, dans ce cas, le contrôle de cohérence appliqué à la réponse générée plutôt qu’un filtre en amont.

Justification. Un motif lexical de domaine (par exemple la présence du mot “conditions” associé à un code d’institution reconnu) constitue déjà une preuve de pertinence suffisante et plus fiable, pour ces intentions, que le score sémantique d’un cross-encoder pénalisé par un décalage de langue entre la question et le texte indexé.

Vérification. Les deux cas régressés (crèche à Salé, FAQ sur l’aide aux oeuvres sociales) obtiennent de nouveau une réponse ancrée sur le contexte réel, sans réintroduire le faux-positif initial sur la question hors-domaine.

Incident – Couverture Centre↔Service manquante pour deux catégories de population

Symptôme observé. Un audit exhaustif de la couverture des données (comparaison systématique du contenu de Neo4j/Qdrant avec les fichiers sources) a révélé une couverture de 0% des relations Centre→Service pour les catégories “personnes âgées” et “personnes en situation de handicap” : aucun centre de ces catégories n’était relié à ses services alors que les données sources les décrivaient bien.

Diagnostic. Le champ `personnes_cibles` des centres normalisés ne contient, dans les données sources, que cinq valeurs distinctes, toutes des libellés français *abrévés* (par exemple “Pers. en sit. hand.”, “Pers. âgés en sit. diff.”) qui ne correspondaient à aucun des motifs de correspondance génériques utilisés par le carnet 03_Match (lesquels cherchaient soit le code de catégorie, soit un libellé complet en arabe ou en français). Une fois ce premier correctif appliqué, un second défaut est apparu immédiatement : lorsque des centaines de centres obtenaient exactement le même score de correspondance (0,75), le tri stable de Python sélectionnait systématiquement les mêmes premiers services par ordre d’apparition dans le fichier, privant durablement les services suivants de toute correspondance.

Solution appliquée. Deux corrections successives, appliquées en amont dans les fichiers intermédiaires puis rejouées à travers le pipeline complet (conformément à la règle d’idempotence du carnet 05, section précédente) : d’abord une table de correspondance explicite entre les cinq libellés abrégés et les catégories de population canoniques ; ensuite un départage déterministe par hachage (couple centre-service) dans le tri des candidats à score égal, pour que chaque service ait une chance réelle d’être apparié plutôt que de dépendre de son ordre d’apparition dans le fichier.

Justification. Une table de correspondance explicite est la solution la plus directe et la plus vérifiable pour un ensemble fermé et connu de cinq valeurs ; le départage par hachage garantit une répartition reproductible et indépendante de l’ordre du fichier source, sans favoriser arbitrairement les premières entrées.

Vérification. Après rechargement complet du graphe, le nombre de relations Centre→Service est passé de 21044 à 24518, avec zéro service orphelin (0 service sans aucun centre associé), vérifié par une requête de comptage exhaustive sur le graphe rechargé.

Incident – Ancrage géographique insuffisant

Symptôme observé. Une question mentionnant explicitement une ville (“je cherche un centre... j’habite à Rabat”) ne faisait remonter qu’un seul centre réellement situé dans la bonne région, le reste des résultats étant dispersé ailleurs sur le territoire.

Diagnostic. Deux causes distinctes, découvertes successivement. D’abord, une formulation comme “je cherche un centre...” en arabe ne correspondait à aucun motif de l’intention *recherche de centre* et retombait sur *question ouverte*, dont le plan d’outils ne prévoit aucune expansion de graphe : l’entité région pourtant correctement extraite ne se traduisait donc par aucun signal géographique. Une fois ce premier motif ajouté, le mécanisme d’expansion géographique existant restait indirect : il partait du voisinage d’un centre déjà trouvé par la recherche vectorielle, et ratait donc la bonne région si aucun résultat vectoriel initial ne s’y trouvait déjà.

Solution appliquée. Ajout des motifs d’intention manquants, puis construction d’un mécanisme d’ancrage géographique *direct*, indépendant du plan d’outils de l’intention et de ce que la recherche vectorielle a trouvé en premier : un résolveur (`geo_resolver.py`) associe les variantes arabes et françaises des noms de ville aux régions canoniques du graphe, et une nouvelle méthode (`expand_by_region`) interroge directement Neo4j pour tous les centres de cette région exacte, injectés comme candidats supplémentaires avec un signal géographique fort. Un correctif de suivi a ensuite ajouté un filtrage par type de population sur cette expansion : la première version faisait en effet remonter n’importe quel centre de la région sans tenir compte de la population ciblée par la question (par exemple un centre de jeunesse pour une question sur les personnes âgées).

Justification. L’ancrage direct par nom de région exact, plutôt que par voisinage d’un résultat vectoriel préexistant, est la seule approche qui garantit qu’une région correctement identifiée dans la question se traduise systématiquement en candidats de cette région, indépendamment de la qualité du classement vectoriel initial.

Vérification. Rejeu de la question originale et de variantes : les centres remontés appartiennent désormais à la bonne région, triés en priorité selon la correspondance de population.

Incident – Fuite de caractères chinois dans les réponses (artefact du modèle)

Symptôme observé. Deux réponses par ailleurs correctes en arabe contenaient des fragments de caractères chinois insérés au milieu ou en fin de texte (par exemple un en-tête de section ou une phrase de clôture), observés indépendamment sur deux questions distinctes lors des tests de bout en bout.

Diagnostic. Un artefact connu de certains modèles de la famille Qwen2.5, qui peut occasionnellement produire des caractères hors de la langue attendue. Le contrôle de langue existant ne testait que les proportions de caractères arabes et latins : un caractère chinois n’étant ni l’un ni l’autre, il n’était tout simplement pas détecté.

Solution appliquée. Ajout d’une détection explicite par plage Unicode (idéogrammes CJK) dans le contrôle de garde-fou et dans le contrôleur de correspondance de langue : la présence du moindre caractère de cette plage invalide systématiquement la réponse, quelle que soit la langue attendue.

Justification. Un caractère chinois n’est jamais légitime dans ce contexte applicatif (bilingue arabe/français uniquement) : sa seule présence est un signal fiable et sans ambiguïté d’un artefact du modèle, ce qui justifie une règle de rejet inconditionnelle plutôt qu’un seuil.

Vérification. Les deux cas ayant révélé le défaut ont été rejoués : la réponse est soit régénérée proprement (retenter avec une directive de langue renforcée), soit remplacée par la réponse de repli si le problème persiste, sans jamais laisser passer de caractères parasites.

Incident – Contrôle de cohérence de langue trop permissif

Symptôme observé. Le garde-fou de langue validait une réponse dès lors qu'elle contenait *un minimum* de caractères reconnaissables (arabe ou latin), sans vérifier que la langue *dominante* correspondait réellement à celle attendue.

Diagnostic. Le test initial ($ar_ratio < 0.05$ and $fr_ratio < 0.05$) ne rejetait qu'une réponse quasiment vide de tout contenu linguistique reconnaissable – il ne comparait jamais les deux proportions entre elles, et laissait donc passer une réponse majoritairement dans la mauvaise langue tant qu'elle contenait quelques mots de la langue attendue.

Solution appliquée. Remplacement par un test comparatif : la proportion de caractères de la langue attendue doit être supérieure ou égale à celle de l'autre langue. En cas d'échec côté synchrone (/chat), une nouvelle génération est retentée avec une directive de langue renforcée ajoutée au prompt; côté flux (/chat/stream), où un nouvel essai après coup casserait le principe même du flux continu, cette directive renforcée est désormais incluse systématiquement dès le premier essai.

Justification. Un test comparatif entre les deux proportions capture directement la notion de "langue dominante", ce qu'un simple seuil minimal ne peut pas faire par construction.

Vérification. Suivi de la proportion de réponses rejetées pour incohérence de langue sur les tests de bout en bout après correctif : plus aucun cas de réponse majoritairement dans la mauvaise langue observé.

Incident – Scripts de démarrage annonçant un succès inexistant

Symptôme observé. Après un cycle rapide d'arrêt puis de redémarrage de la pile applicative, le script de démarrage affichait un message de succès ("prêt") alors que le serveur llama.cpp n'avait en réalité jamais réussi à se lier à son port – l'API fonctionnait dans un état dégradé non détecté, jusqu'à l'échec d'une vraie question utilisateur.

Diagnostic. La boucle de vérification de disponibilité du script s'épuisait silencieusement sans jamais confirmer un succès réel, mais le message de fin de script restait inconditionnel : le script ne distinguait pas "la boucle s'est terminée" de "la boucle a confirmé un succès".

Solution appliquée. Réécriture des scripts de démarrage et d'arrêt : chaque étape vérifie désormais un succès réel avant de continuer (et arrête le script avec le journal correspondant en cas d'échec), la vérification finale interroge le corps JSON du point de santé de l'API plutôt qu'un simple code HTTP 200, et le script d'arrêt attend activement la fin réelle des processus avant de rendre la main.

Justification. Un script d'orchestration qui ne vérifie pas réellement ce qu'il annonce est pire qu'une absence de script : il crée une fausse confiance qui retarde la détection d'une panne réelle.

Vérification. Cycles d'arrêt puis de redémarrage forcés répétés après correctif : le script échoue désormais explicitement, avec le journal pertinent, dans les cas où l'ancienne version aurait annoncé un succès trompeur.

5.1 Mémoire conversationnelle

Incident – Perte totale de contexte sur une question de suivi elliptique

Symptôme observé. Une première question sur la rééducation motrice pour les personnes en situation de handicap, posée via les centres CPSH (centres de promotion sociale des handicapés), obtenait une bonne réponse, correctement ancrée sur le contexte. La question de suivi, formulée de façon elliptique (*“et quelles sont les conditions et les documents requis ?”*, sans reformuler le sujet), obtenait une réponse portant sur des conditions de catégories de service totalement différentes – le système avait entièrement perdu le fil de la conversation.

Diagnostic. Chaque requête était traitée de façon strictement indépendante : aucune information sur les tours précédents de la même session n’était conservée ni transmise au pipeline de récupération. Une question de suivi courte, sans entité métier propre (aucune population, institution ou région mentionnée explicitement), repartait donc sur une recherche vectorielle générique déconnectée du sujet réel.

Solution appliquée. Six éléments complémentaires, conçus ensemble et validés par un plan présenté et approuvé avant implémentation :

1. Le widget génère un identifiant de session (UUID) persisté en `localStorage` du navigateur, transmis à chaque appel de conversation – stable sur toute la visite, y compris après un rafraîchissement de page.
2. Un nouveau module (`session_memory.py`) tient, côté Redis, un historique glissant par session (jusqu’à quatre tours, expiration glissante de trente minutes), chaque tour enregistrant la question, l’intention détectée, les entités extraites et un extrait de la réponse.
3. Une heuristique de détection de suivi (question courte, huit mots ou moins, et absence de toute entité propre extraite) déclenche une fusion de la requête actuelle avec celle du tour précédent pour la seule étape de récupération – la question originale reste seule utilisée pour l’affichage et pour le prompt final envoyé au modèle.
4. Les entités du tour précédent sont héritées quand le tour actuel n’en extrait aucune (par exemple, la population “personnes en situation de handicap” identifiée au premier tour reste active pour la question de suivi).
5. Un court bloc d’historique (les deux derniers tours) est injecté dans le prompt envoyé au modèle de langage, pour que la génération elle-même reste consciente du fil de la conversation.
6. La clé de cache de réponse devient sensible à la session pour les tours identifiés comme des suivis, afin d’éviter qu’une question de suivi générique (*“et quels documents ?”*) ne retourne, via le cache partagé, la réponse d’une autre session portant sur un sujet totalement différent.

Justification. Cette conception distingue délibérément deux notions : “ce qui est cherché” (la requête enrichie, utilisée uniquement pour orienter la récupération) et “ce à quoi il faut répondre” (la question originale de l’utilisateur, seule montrée au modèle comme instruction finale). Cette séparation évite qu’une fusion de contexte ne transforme silencieusement la question posée par l’utilisateur.

Vérification. Le scénario exact ayant révélé le défaut a été rejoué de bout en bout après implémentation, avec un identifiant de session fixe : la question de suivi reste désormais centrée sur la rééducation motrice et les centres CPSH, avec des sources cohérentes avec le premier tour. Le mécanisme a été vérifié une seconde fois sur un sujet entièrement différent (formation professionnelle pour les femmes en situation difficile), confirmant que le com-

portement se généralise : l’inspection directe de la clé Redis de session a montré l’héritage correct de l’entité de population entre les deux tours dans les deux cas.

5.2 Comparaison de modèles (Qwen2.5-7B contre Gemma-3-4B)

Face à la latence importante des réponses ancrées (60 à plus de 300 secondes selon la charge du CPU partagé), un test comparatif a été mené entre le modèle de production (Qwen2.5-7B-Instruct) et une alternative plus légère (Gemma-3-4B-it), sur un même contexte réel et un même prompt, chacun exposé sur un port distinct pour ne jamais interrompre le service en production. Le résultat mesuré : un débit d’environ 3,64 tokens par seconde pour Gemma-3-4B contre environ 2,00 pour Qwen2.5-7B sur le même test – un gain de l’ordre de 1,8 fois – pour une qualité de réponse ancrée jugée comparable sur l’exemple testé. La décision, explicitement prise par le porteur du projet, a été de conserver Qwen2.5-7B en production pour le moment : ce test reste documenté comme une option de repli si la latence devenait le facteur limitant principal de l’expérience utilisateur (voir recommandations, section 7).

5.3 Réponse de repli inutilisable pour les foires aux questions et les programmes

Incident – La réponse de repli du garde-fou n’affichait aucun contenu réel pour les FAQ et les programmes

Symptôme observé. Sur un jeu de quinze questions diverses représentatives de formulations citoyennes réelles (registre formel et familial, arabe et français, questions directes et indirectes), trois questions – portant sur les conditions de l’aide aux oeuvres sociales, sur le budget du programme de création de crèches sociales 2027, et sur le budget du programme des centres de jour pour personnes âgées – ont reçu, au lieu d’une réponse, une simple liste de titres de FAQ ou de programmes répétés, sans jamais citer le contenu réel ni le montant du budget, alors que ces informations étaient bien présentes dans le contexte récupéré (28 millions de MAD pour les crèches, 18 millions de MAD pour les centres de jour, vérifiés directement dans Neo4j).

Diagnostic. Ces trois questions avaient déclenché le mécanisme de repli du garde-fou (la réponse générée par le modèle avait été jugée invalide et remplacée par un résumé structuré du contexte brut). Or la fonction de repli ne lisait que les champs propres aux résultats de type Centre (adresse, conditions) – jamais le champ de texte détaillé (qui, pour une FAQ, contient la question *et* la réponse concaténées) ni le champ budget (propre aux programmes). Un second défaut, plus discret, a été relevé au passage : l’en-tête de cette réponse de repli était codé en dur en bilingue arabe/français, y compris pour une question posée entièrement en français.

Solution appliquée. La fonction de repli affiche désormais le contenu textuel réel et le budget, quand ils existent, pour chaque résultat listé, en plus des champs déjà couverts (adresse, conditions, documents) – et son en-tête est désormais choisi selon la langue de la question plutôt que codé en dur.

Justification. Le champ de texte détaillé et le champ budget étaient déjà extraits et disponibles dans la structure de contexte (calculés par le *Context Builder*, section 1) – il ne s’agissait pas d’aller chercher une nouvelle donnée, mais de cesser d’ignorer une donnée déjà présente. Localiser l’en-tête selon la langue de la question est cohérent avec l’exigence de bilinguisme réel énoncée en introduction.

Vérification. Vérification en deux temps : un test unitaire direct de la fonction corrigée sur un contexte synthétique contenant un programme (avec budget) et une FAQ, confirmant l’affichage du texte et du budget avec l’en-tête dans la bonne langue ; puis un rejeu réel des trois questions originales, qui citent désormais les montants exacts de 28, 35 et 18 millions de MAD lorsque le mécanisme de repli se déclenche à nouveau.

6 Méthodologie de test et de validation

La fiabilité du système repose autant sur la méthode de vérification employée tout au long du projet que sur les correctifs eux-mêmes. Trois principes ont été appliqués de façon constante.

L’inspection directe des données prévaut toujours sur la supposition. Avant de conclure qu’une réponse est correcte ou incorrecte, le contenu réel de Neo4j et de Qdrant a systématiquement été interrogé pour établir une vérité de référence – y compris lorsque cela signifiait reproduire fidèlement une coquille présente dans les données sources plutôt que de la “corriger” silencieusement dans la réponse générée. Un audit exhaustif de complétude a par ailleurs été mené en comparant, entité par entité, le contenu des deux bases aux fichiers sources d’origine, ce qui a directement mené à la découverte de l’incident de couverture Centre-Service décrit en section 5.

Chaque correctif est vérifié par un nouveau test réel, jamais supposé résolu sur la seule lecture du code. Après chaque modification, la pile complète est redémarrée (arrêt puis démarrage, jamais un redémarrage partiel) et le cache Redis est systématiquement vidé avant de rejouer le scénario ayant révélé le défaut, pour exclure qu’une ancienne réponse mise en cache ne masque une régression ou ne simule à tort une correction.

Les tests couvrent délibérément la diversité réelle des formulations citoyennes. Au-delà des scénarios de démonstration, un jeu de quinze questions a été construit pour représenter la variété réelle des façons de poser une question : registre formel et darija familière, arabe et français, questions directes et questions elliptiques ou vagues, questions à critères multiples, question de suivi conversationnel, question hors domaine, calcul arithmétique, et un cas de négation (limitation documentée ci-dessous). Sur ce jeu, treize questions sur quinze ont abouti à une réponse jugée correcte et utile après comparaison avec le contenu réel de la base ; les défauts identifiés (réponse de repli inexploitable pour les FAQ/programmes, une formulation familière de demande de crèche mal reconnue, la limitation de négation, un dépassement de délai sur une question à critères multiples) ont chacun été soit corrigés et re-vérifiés (cas de la réponse de repli, section 5), soit documentés comme limitation connue plutôt que dissimulés.

7 Limitations connues et recommandations

Certaines limites ont été identifiées mais délibérément documentées plutôt que corrigées dans le cadre de ce projet, soit parce que leur correction dépasse le périmètre d’un ajustement ciblé, soit parce qu’elles relèvent d’un arbitrage produit qui n’a pas encore été tranché.

La négation n’est pas comprise par la couche de récupération sémantique. Une question formulée comme “quels centres n’acceptent *pas* les enfants de moins de trois ans ?” obtient une réponse portant sur les centres qui *acceptent* cette tranche d’âge – l’exact inverse de l’intention. La recherche vectorielle et le classement métier actuels ne modélisent pas la négation logique ; ce cas a été observé et reproduit dans le jeu de quinze questions de test, et reste documenté comme limitation connue plutôt que corrigé, une solution robuste nécessitant une étape de compréhension de la question en amont de la récupération.

Le calcul arithmétique confié au modèle de langage sur de grands nombres n’est pas

fiable. Le calculateur métier couvre les opérations simples détectées explicitement, mais toute opération numérique laissée à la charge du modèle de langage lui-même (par exemple une somme demandée implicitement au sein d’une réponse plus large) reste sujette à erreur, comme tout grand modèle de langage généraliste. La recommandation est de systématiser le calcul en code pour toute opération numérique identifiable, plutôt que de faire confiance au modèle.

La latence reste la contrainte dominante de l’expérience utilisateur. Sur ce serveur CPU partagé, une réponse ancrée sur le contexte prend de 60 à plus de 300 secondes selon la contention, avec une variabilité significative observée en conditions réelles (débit mesuré entre 1,3 et 4 tokens par seconde selon la charge concurrente). Le test comparatif avec Gemma-3-4B (section 5.1) offre une option de repli si cette latence devenait inacceptable pour les utilisateurs, mais n’a pas été retenue à ce stade sur décision explicite du porteur de projet.

L’absence de superviseur de processus pour la couche applicative. À plusieurs reprises durant le développement, le serveur `llama.cpp`, l’API et le serveur du widget se sont arrêtés entre deux sessions de travail (les conteneurs de stockage, gérés séparément avec redémarrage automatique, n’étant pas affectés), nécessitant un redémarrage manuel. La mise en place d’un superviseur de processus (par exemple `systemd`) pour la couche applicative est recommandée pour la mise en production définitive.

Absence de dépôt de gestion de versions. L’ensemble des modifications décrites dans ce rapport a été appliqué directement sur les fichiers du projet, sans historique de versions (`git`) – un choix délibéré maintenu jusqu’ici, mais qui limite la traçabilité fine et la possibilité de revenir à un état antérieur précis. L’initialisation d’un dépôt est recommandée pour la suite du projet.

8 Conclusion

Ce projet illustre le passage d’un prototype GraphRAG fonctionnel à un système dont la fiabilité a été construite incident par incident, chacun découvert par un test réel et corrigé à sa racine plutôt que contourné. L’architecture retenue – routage d’intention, planification d’outils, récupération hybride Neo4j/Qdrant, fusion pondérée par des règles métier explicites, reclassement sémantique, filtre de pertinence adapté à chaque type de preuve disponible, garde-fou de sortie, et désormais mémoire conversationnelle par session – traduit directement les trois exigences fixées au départ : exactitude vérifiable, bilinguisme réel, et fonctionnement entièrement local sur une infrastructure CPU maîtrisée.

Les corrections apportées ne relèvent pas d’ajustements cosmétiques : un filtre de pertinence structurellement incapable de rejeter une question hors domaine, une couverture de données nulle sur deux catégories de population entières, ou une réponse de repli qui omettait le contenu réel des foires aux questions et des programmes, sont autant de défauts qui, non corrigés, auraient directement compromis la confiance que l’exigence d’exactitude cherchait précisément à garantir. Leur résolution méthodique, documentée dans ce rapport avec le symptôme, le diagnostic, la solution et la vérification propres à chacun, constitue la meilleure garantie apportée à ce jour de la fiabilité du système.

Les limitations qui subsistent – gestion de la négation, latence sur ce CPU partagé, absence de superviseur de processus et de gestion de versions – sont explicitement documentées plutôt que dissimulées, et constituent la feuille de route naturelle de la prochaine itération du projet.